Pontifícia Universidade Católica de Minas Gerais
Disciplina: Teste de Software
Professor: Cleiton Tavares

Refatoração de Testes e Detecção de Test Smells

Análise com ESLint e refatoração da suíte de testes do *UserService*

Aluno: Pedro de Sousa Motta
Repositório: https://github.com/Psmotta/test-smelly

Belo Horizonte, 2026

Este relatório apresenta a análise e a refatoração da suíte de testes do `UserService`, feitas em duas frentes: leitura manual do código e detecção automática com o ESLint e o plugin `eslint-plugin-jest`. A suíte refatorada está em `__tests__/userService.clean.test.js`, e o arquivo original não foi alterado.

1. Análise de Smells

Os smells abaixo foram encontrados na leitura de `test/userService.smelly.test.js`, antes de executar o ESLint.

1.1 Lógica Condicional no Teste (Conditional Test Logic)

Onde: "deve desativar usuários se eles não forem administradores" escolhe os `expect` com `for` e `if/else` (código na Seção 2.1), e "deve falhar ao criar usuário menor de idade" coloca o `expect` dentro de um `catch`:

```
try {
  userService.createUser('Menor', 'menor@email.com', 17);
} catch (e) {
  expect(e.message).toBe('O usuário deve ser maior de idade.');
}
```

Por que é um mau cheiro: o teste deixa de ter um caminho único. O leitor precisa simular a execução para saber quais verificações rodam, e o próprio teste passa a ter lógica sujeita a erros.

Risco: verificações que podem não executar. Se `createUser` parar de lançar a exceção, o `catch` não roda, nenhum `expect` é avaliado e o teste passa. É um falso positivo que esconde a quebra da regra de idade mínima, como comprovado na Seção 2.3.

1.2 Teste Frágil (Fragile Test / Sensitive Equality)

Onde: "deve gerar um relatório de usuários formatado":

```
const linhaEsperada = `ID: ${usuario1.id}, Nome: Alice, Status: ativo\n`;
expect(relatorio).toContain(linhaEsperada);
expect(relatorio.startsWith('--- Relatório de Usuários ---')).toBe(true);
```

Por que é um mau cheiro: o teste compara a string exata do relatório, com separadores, ordem dos campos, quebra de linha e cabeçalho com hífens. Ele verifica a implementação da formatação, e não o comportamento, que é listar os usuários com seus dados. O próprio `userService.js` avisa que esse formato pode mudar.

Risco: falsos alarmes. Uma mudança legítima de layout quebra o teste sem haver bug, e a equipe se acostuma a "consertar o teste" em vez de confiar nele.

1.3 Teste Ansioso (Eager Test)

Onde: "deve criar e buscar um usuário corretamente" tem dois Acts, marcados pelos próprios comentários `// Act 1: Criar` e `// Act 2: Buscar`. O teste de desativação também cobre dois cenários de uma vez, o usuário comum e o administrador.

Por que é um mau cheiro: cada teste deveria verificar um único comportamento. Ao juntar vários, o nome fica genérico ("corretamente") e o padrão AAA se perde.

Risco: a falha não diz qual comportamento quebrou, e a primeira falha esconde as seguintes. Os cenários também ficam verificados pela metade: para o administrador, o original só confere o retorno `false`, e não se o status continuou `'ativo'`.

1.4 Outros smells observados

- **Teste Ignorado (Ignored Test):** o `test.skip` com `TODO` e corpo vazio deixa o relatório sem usuários sem nenhuma verificação, e o lembrete tende a ser esquecido.
- **Convidado Misterioso (Mystery Guest), em forma leve:** `dadosUsuarioPadrao` fica no topo do arquivo, longe do único teste que o usa, e obriga o leitor a procurar os dados para entender a verificação.

2. Processo de Refatoração

2.1 Teste escolhido: “deve desativar usuários se eles não forem administradores”

Foi escolhido por reunir Lógica Condicional e Eager Test no mesmo teste.

ANTES · test/userService.smelly.test.js

```
test('deve desativar usuários se eles não forem administradores', () => {
  const usuarioComum = userService.createUser('Comum', 'comum@teste.com', 30);
  const usuarioAdmin = userService.createUser('Admin', 'admin@teste.com', 40, true);

  const todosOsUsuarios = [usuarioComum, usuarioAdmin];

  // O teste tem um loop e um if, tornando-o complexo e menos claro.
  for (const user of todosOsUsuarios) {
    const resultado = userService.deactivateUser(user.id);
    if (!user.isAdmin) {
      // Este expect só roda para o usuário comum.
      expect(resultado).toBe(true);
      const usuarioAtualizado = userService.getUserById(user.id);
      expect(usuarioAtualizado.status).toBe('inativo');
    } else {
      // E este só roda para o admin.
      expect(resultado).toBe(false);
    }
  }
});
```

DEPOIS · __tests__/userService.clean.test.js

```
describe('deactivateUser', () => {
  test('deve desativar um usuário comum e retornar true', () => {
    // Arrange
    const usuarioComum = userService.createUser('Comum', 'comum@teste.com', 30);

    // Act
    const resultado = userService.deactivateUser(usuarioComum.id);

    // Assert
    expect(resultado).toBe(true);
    expect(userService.getUserById(usuarioComum.id).status).toBe('inativo');
  });

  test('não deve desativar um usuário administrador', () => {
    // Arrange
    const usuarioAdmin = userService.createUser('Admin', 'admin@teste.com', 40, true);

    // Act
    const resultado = userService.deactivateUser(usuarioAdmin.id);

    // Assert
    expect(resultado).toBe(false);
    expect(userService.getUserById(usuarioAdmin.id).status).toBe('ativo');
  });

  test('deve retornar false ao tentar desativar um usuário inexistente', () => {
    // Arrange
    const idInexistente = 'id-que-nao-existe';

    // Act
    const resultado = userService.deactivateUser(idInexistente);

    // Assert
    expect(resultado).toBe(false);
  });
});
```

Decisões e efeito sobre os smells:

1. **Um comportamento por teste:** o teste virou três, cada um com um único Act. Isso elimina o Eager Test, e a falha aponta o cenário exato.
2. **Nenhuma lógica no teste:** sem `for` e `if/else`, todo `expect` executa. Isso elimina a Lógica Condicional e os três erros `jest/no-conditional-expect` do teste.
3. **AAA explícito e nomes descritivos:** blocos Arrange, Act e Assert marcados, dados criados no próprio teste e nomes que dizem o cenário e o resultado esperado.
4. **Verificação mais forte:** o teste do administrador confere também que o status continua `'ativo'`, o que o original não fazia.

2.2 Demais testes da suíte

Teste original	Smell	Como ficou no arquivo limpo
deve criar e buscar um usuário corretamente	Eager Test, Convidado Misterioso	Um teste para `createUser` e outro para `getUserById`, com os dados no Arrange.
deve gerar um relatório de usuários formatado	Teste Frágil	`toContain` do id e do nome de cada usuário e, em outro teste, do status `inativo` após a desativação, sem depender do layout.
deve falhar ao criar usuário menor de idade	Lógica Condicional (`try/catch`)	`expect(criarUsuario).toThrow(...)`, que falha se nada for lançado, e um teste do limite de 18 anos.
deve retornar uma lista vazia quando não há usuários	Teste Ignorado (`test.skip`)	Implementado como "deve informar no relatório quando não há usuários cadastrados".

Com testes extras de campos obrigatórios e ids inexistentes, a suíte limpa tem 14 testes, um `describe` por método.

2.3 Evidência: a suíte limpa detecta o que a original deixa passar

Cada alteração foi aplicada sozinha em `src/userService.js` e desfeita após rodar as duas suítes.

Alteração no código de produção	É bug?	Suíte original	Suíte limpa
Remover a validação de idade mínima	Sim	passa (não detecta)	falha (detecta)
Trocar `idade < 18` por `idade <= 18`	Sim	passa (não detecta)	falha (detecta)
Alterar o status antes de checar se o usuário é administrador	Sim	passa (não detecta)	falha (detecta)
Remover a mensagem do relatório vazio	Sim	passa (não detecta)	falha (detecta)
Trocar o separador `", "` por `" | "` nas linhas do relatório	Não	falha (falso alarme)	passa

A suíte original deixou passar os quatro bugs e falhou na única mudança inofensiva. A limpa acertou os cinco.

3. Relatório da Ferramenta (ESLint)

Foram instalados o `eslint` e o `eslint-plugin-jest` e criado o `.eslintrc.json` do enunciado. O ESLint ficou na versão 8.57.1 porque, a partir da versão 9, esse formato deixou de ser o padrão.

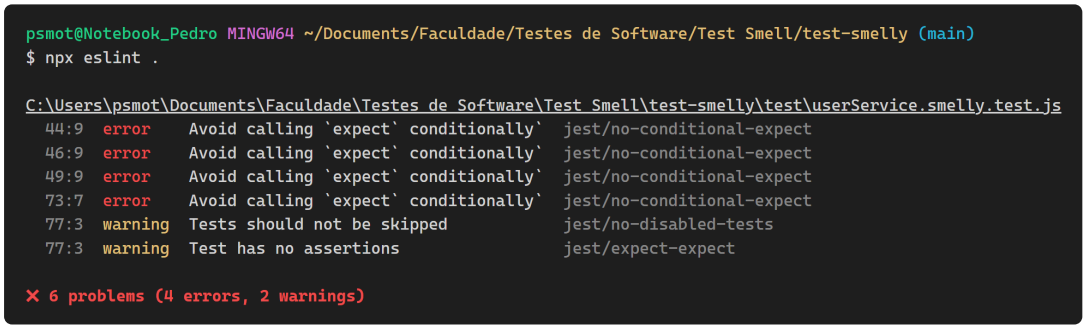


Figura 1 – Saída de `npx eslint .` na primeira execução: 6 problemas (4 erros e 2 avisos).

- **4 erros** `jest/no-conditional-expect` (linhas 44, 46 e 49 no `if/else` e 73 no `catch`) confirmam a Lógica Condicional da Seção 1.1, inclusive a variante com `try/catch`.
- **2 avisos na linha 77**, `jest/no-disabled-tests` e `jest/expect-expect`, apontam o Teste Ignorado e a ausência de asserções nele. A severidade segue o `.eslintrc.json`: o expect condicional é `error`, pois pode esconder bugs, e o teste desabilitado é `warn`.

Comparada à análise manual, a ferramenta achou em segundos, com linha e coluna exatas, os smells de padrão sintático. Ela não aponta o Teste Frágil, o Eager Test nem o Convidado Misterioso, que dependem da intenção do teste e exigem análise humana. Por ser automática, pode rodar a cada commit ou no CI. Após a refatoração, o ESLint não aponta nenhum erro ou aviso no arquivo limpo, e `npm test` aprova as duas suítes: 18 testes passam e o único ignorado é o `test.skip` do arquivo original, mantido sem alterações.

4. Conclusão

O trabalho de Teste de Mutação mostrou que uma suíte pode passar sem ser eficaz, e os test smells explicam parte disso: a suíte original passava inteira, mas deixou passar quatro bugs e acusou uma mudança inofensiva. Testes limpos, com AAA, um comportamento por teste e verificações de comportamento, falham só quando há bug e dizem o que quebrou. Isso melhora a qualidade, porque os defeitos aparecem cedo, e a sustentabilidade, porque testes legíveis custam menos para manter e dão segurança para evoluir o código. A análise estática completa esse cuidado: o ESLint barra automaticamente os smells de padrão sintático a cada commit, e a revisão humana cuida dos smells de intenção.